

Аналитический отчёт по проекту:

Архиватор на основе алгоритма Хаффмана

Долженко Александра Борисовна

Руководитель: Юрий Викторович Литвинов

1 Введение

В рамках учебной практики была разработана программа для сжатия и восстановления файлов с использованием алгоритма Хаффмана. Алгоритм Хаффмана — это метод сжатия без потерь, основанный на построении оптимального префиксного кода для символов входного потока.

Цель работы: реализовать консольное приложение, выполняющее сжатие и разжатие файлов, а также провести исследование его эффективности.

2 Архитектура программы

Программа состоит из следующих модулей:

- **bitOperation** — работа с битовым буфером (запись/чтение битов);
- **queue** — очередь с приоритетом (куча) для построения дерева Хаффмана;
- **huffmanTree** — построение дерева и генерация кодов;
- **huffman** — основная логика сжатия и распаковки.

2.1 Алгоритм сжатия

1. **Подсчёт частот:** входной файл читается побайтово, для каждого из 256 возможных символов подсчитывается частота встречаемости.
2. **Построение дерева Хаффмана:** на основе частот строится бинарное дерево, где листьями являются символы, а внутренние узлы объединяют два наименее частых поддерева. Для эффективного построения используется очередь с приоритетом (куча).
3. **Генерация кодов:** дерево обходится рекурсивно. При переходе в левое поддерево к коду дописывается 0, в правое — 1. Коды сохраняются в узлах-листьях.
4. **Кодирование:** исходный файл читается повторно, каждый символ заменяется на его битовый код. Коды упаковываются в байты с использованием битового буфера.
5. **Сохранение служебной информации:** в выходной файл записываются маркер, количество уникальных символов, пары (символ, частота), размер исходного файла, а затем — сжатые битовые данные.

2.2 Алгоритм распаковки

1. **Чтение заголовка:** из сжатого файла считываются маркер, количество уникальных символов, пары (символ, частота) и размер исходного файла.
2. **Восстановление дерева:** по прочитанным частотам заново строится дерево Хаффмана (аналогично этапу сжатия).
3. **Декодирование:** биты читаются по одному, начиная от корня дерева. При чтении 0 выполняется переход в левое поддерево, при 1 — в правое.
4. **Вывод символа:** при достижении листа соответствующий символ записывается в выходной файл. Процесс повторяется, пока не будет восстановлено ровно столько байт, сколько указано в заголовке.

3 Формат сжатого файла

Сжатый файл имеет следующую структуру.

3.1 Маркер

Первый байт файла всегда равен `0xFF`. Это значение используется как сигнатура формата: при распаковке программа проверяет, что первый байт равен `0xFF`, и завершается с ошибкой в противном случае.

3.2 Количество уникальных символов

После маркера записывается целое число, указывающее, сколько различных символов встречается в исходном файле. Это значение определяет, сколько пар «символ — частота» будет записано далее.

3.3 Таблица частот

Для каждого уникального символа в файл записывается пара значений:

- `symbol` — значение байта (один байт, диапазон 0–255);
- `frequency` — частота встречаемости данного символа в исходном файле (восемь байт, тип `uint64_t`).

3.4 Исходный размер

Следующим полем записывается размер исходного файла в байтах (тип `uint64_t`, 8 байт). Это значение используется при распаковке для контроля: процесс декодирования завершается после восстановления указанного количества байт.

3.5 Сжатые данные

Последняя часть файла — это битовый поток, содержащий коды Хаффмана для каждого символа исходного файла. Биты упаковываются в байты (по 8 бит в каждом). Последний байт может быть неполным; лишние биты игнорируются при распаковке.

3.6 Обработка файла с одним символом

Обработка случая с одним уникальным символом выполняется на уровне построения дерева. Если в таблице частот присутствует только один символ, функция `buildTree()` создаёт фиктивный родительский узел. Единственный символ получает код '0' (длиной 1 бит), а дерево состоит из двух узлов: корня и листа.

4 Результаты тестирования

4.1 Тесты в репозитории

В репозитории находятся 5 тестовых файлов с текстом различного рода: файл из множества Hello World, повторяющийся символ, отрывки из художественного произведения на русском и на английском языке, а также небольшой файл, где были совмещены сразу три языка, включая китайский, чтобы проверить работоспособность алгоритма с иероглифами. Все тесты пройдены успешно.

Содержание	Исходный (В)	Сжатый (В)	Экономия (%)
Hello world	863	427	50.52
Повторяющийся символ	58,818	7,384	87.45
Английский текст	8,802	5,556	36.88
Русский текст	3,711	2,392	35.55
Несколько языков	3,933	3,593	8.65

4.2 Анализ степени сжатия

Для более детального анализа были созданы дополнительные тестовые файлы различных типов.

Тип данных	Исходный (В)	Сжатый (В)	Экономия (%)
Текст (EN)	58	251	-332.75
Текст (EN)	102,400	60,417	41.00
Текст (RU)	3,711	2,392	35.55
Повторяющийся символ	10,000	1,281	87.19
Нулевые данные	102,400	12,822	87.48
Случайные данные	102,400	104,717	-2.26
Изображение	102,400	104,717	-2.26

5 Анализ производительности

5.1 Время выполнения для тестов в репозитории

Тест	Размер (В)	Время (сек)
Hello world	863	0.00
Повторяющийся символ	58,818	0.00
Английский текст	8,802	0.00
Русский текст	3,711	0.00
Несколько языков	3,933	0.00

Все тесты на файлах в репозитории выполняются за < 1 мс, так как размер файлов слишком мал для измерения.

5.2 Время выполнения для файлов различных размеров

Для оценки производительности алгоритма были созданы файлы различных размеров, включая большие файлы (до 50 МБ), и измерено время их сжатия.

Тип данных	Размер	Время (сек)
Маленькие файлы		
Очень маленький текст	0.05 KB	0.00
Текст (EN)	1 KB	0.00
Текст (EN)	10 KB	0.01
Текст (EN)	100 KB	0.04
Изображение	100 KB	0.06
Случайные данные	100 KB	0.08
Нулевые данные	100 KB	0.00
Большие файлы		
Текст (EN)	1 MB	0.20
Текст (EN)	10 MB	2.01
Текст (EN)	50 MB	17.43
Повторяющиеся данные	10 MB	1.25
Нулевые данные	10 MB	0.54
Случайные данные	10 MB	17.96

Как видно из таблицы, время сжатия растёт примерно линейно с увеличением размера файла. Для текстовых файлов скорость сжатия составляет 3–5 МБ/сек для файлов до 10 МБ, однако для файла 50 МБ скорость снижается до 2.9 МБ/сек, что связано с накладными расходами ввода-вывода.

Наиболее быстро сжимаются нулевые данные (10 МБ за 0.54 сек, 18.5 МБ/сек), так из них получается минимальное дерево Хаффмана и коды длиной 1 бит. Повторяющиеся данные также сжимаются быстрее текста (10 МБ за 1.25 сек, 8 МБ/сек) благодаря меньшему числу уникальных символов, большому количеству повторений и более коротким кодам.

Медленнее всего обрабатываются случайные данные (10 МБ за 17.96 сек, 0.56 МБ/сек). Все 256 символов имеют примерно равные частоты, а коды имеют длину 8 бит, сжатия нет или оно становится отрицательным, и алгоритм записывает большой заголовок.

Таким образом, производительность алгоритма существенно зависит не только от размера, но и от структуры данных: наибольшая эффективность достигается на хорошо сжимаемых данных с малым алфавитом, тогда как на случайных данных алгоритм показывает наихудшую скорость.

6 Проверка памяти (Valgrind)

Valgrind не обнаружил утечек памяти:

```

1 ==13166== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from
   0)
2 ==13166== All heap blocks were freed -- no leaks are possible

```

7 Заключение

Разработанный архиватор успешно реализует алгоритм Хаффмана, проходит все тесты и демонстрирует высокую эффективность на текстовых данных.

Параметр	Значение
Утечек памяти	нет
Среднее время сжатия (100 KB)	0.04 сек
Максимальное сжатие	87.48% (нулевые данные)
Минимальное сжатие	-332.75% (очень маленький файл)